QUIZNET – MULTIPLAYER QUIZ MANAGEMENT SYSTEM

Project Report | IT Workshop III | Academic Year 2026

Kolimi Mahesh

2025BCS0077

Table of Contents

1. Abstract
2. Introduction
3. Problem Statement
4. Objectives
5. Technologies Used
6. System Architecture
7. Database Design
8. Methodology and Implementation
9. Testing and Results
10. Advantages
11. Limitations
12. Conclusion

1. Abstract

QuizNet is a multiplayer quiz management system developed using Python. The application allows multiple students to participate in quizzes through a client-server architecture. It uses TCP sockets for communication between clients and the server, while multithreading allows the server to handle multiple client connections concurrently.

MySQL is used to store quiz questions and player results. The application calculates scores, displays leaderboards, and maintains result history. NetworkX is used to represent the system architecture as a graph, while Matplotlib supports graphical visualization.

The main purpose of this project is to demonstrate the practical application of computer networking, concurrent programming, database management, and data visualization in a single system.

2. Introduction

Online quizzes are useful for evaluating students' knowledge and providing immediate feedback. A multiplayer quiz application allows several students to participate in the same quiz while a central server manages the questions, answers, and results.

QuizNet implements this concept using Python. The server communicates with multiple clients through TCP sockets. Threading helps manage concurrent connections, and a MySQL database stores questions and completed quiz results.

The project is developed in a Jupyter Notebook, making it easier to execute the implementation step by step and examine the outputs.

3. Problem Statement

In a traditional quiz system, managing multiple participants, maintaining question records, calculating scores, and displaying results can require considerable manual effort.

The objective of QuizNet is to develop a Python-based quiz management system that allows multiple clients to communicate with a central server, answer multiple-choice questions, receive scores, and access saved results.

The system combines networking, multithreading, database operations, and visualization to provide a practical demonstration of these concepts.

4. Objectives

The primary objectives of the project are:

- To implement client-server communication using TCP sockets.
- To support multiple clients using Python threading.
- To store quiz questions and player results in a MySQL database.

- To provide a mechanism for adding and viewing questions.
- To evaluate submitted answers and calculate scores.
- To display leaderboards and saved result history.
- To represent the system architecture using NetworkX.
- To visualize student performance using Matplotlib.

5. Technologies Used

Technology	Purpose
Python	Main programming language
TCP sockets	Communication between the server and clients
Threading	Handling multiple client connections
MySQL	Persistent storage of questions and results
MySQL Connector/Python	Connecting Python to MySQL
Pandas	Organizing and displaying result data
NetworkX	Creating the system architecture graph
Matplotlib	Plotting graphs and visualizations
Jupyter Notebook	Development and execution environment

6. System Architecture

QuizNet follows a client-server architecture. Multiple student clients communicate with a central Python server, which manages quiz activities and interacts with the database.

Student Client A

Connects to the server and submits answers

Python Quiz Server

Connection handling, questions, scoring and results

MySQL Database

Question records and quiz results

Student Client B and other clients connect to the same server.

6.1 Working Process

1. The server is initialized and connects to the database.
2. Students connect to the server using client connections.
3. The server provides quiz questions and answer options.
4. Students submit their answers through the client.
5. The server evaluates answers and calculates scores.
6. Completed results are stored in MySQL.
7. The application displays the leaderboard and result history.

6.2 Role of Each Component

Client: Provides the interface through which a participant connects and answers questions.

Server: Manages client connections, quiz questions, answer processing, and scoring.

Database: Stores questions and completed quiz results for later retrieval.

NetworkX: Represents the relationships between clients, the server, and the database.

Matplotlib: Supports graphical presentation of information, including student performance.

7. Database Design

The project uses MySQL to store question information and quiz results.

7.1 Questions Table

The questions table stores the information required to conduct a multiple-choice quiz.

Field	Description
Question ID	Unique identifier for a question
Question Text	Text of the question
Option A	First answer choice
Option B	Second answer choice
Option C	Third answer choice
Option D	Fourth answer choice
Correct Option	Correct answer choice
Topic	Category of the question

The `add_question()` function inserts questions into the database and validates that the correct option is A, B, C, or D.

7.2 Results Table

The results table stores information about completed quizzes.

Field	Description
Player Name	Name of the participant
Score	Score obtained
Total Questions	Total number of quiz questions
Percentage	Percentage score
Completed At	Completion date and time

The stored results can be retrieved to display previous performances and leaderboard information.

8. Methodology and Implementation

The project is implemented in Python using separate notebook cells for setup, database operations, quiz execution, result analysis, and visualization.

8.1 Database Initialization

The notebook establishes a connection between Python and MySQL. It initializes the database tables required to store questions and quiz results.

The database connection is used whenever the application needs to insert or retrieve records.

8.2 Question Management

The `add_question()` function accepts the question text, four answer options, the correct option, and the topic.

It validates the correct option before inserting the question into the database. The `view_questions()` function displays stored questions so that the user can verify that they have been added.

8.3 Client-Server Communication

TCP sockets provide a communication channel between the server and clients. The server manages incoming client connections and exchanges quiz information with the participants.

TCP is suitable for this application because it provides reliable, ordered delivery of data between connected programs.

8.4 Multithreading

The Python threading module allows the server to handle multiple client connections concurrently.

Without concurrent handling, a server might have to wait for one client's work to finish before attending to another. Threads help the application serve multiple connected participants more efficiently.

8.5 Quiz Evaluation and Scoring

The server evaluates submitted answers against the correct options and calculates each participant's score. After a quiz is completed, the results can be stored in the database for later analysis.

The leaderboard provides a way to compare participant scores.

8.6 Result Analysis

Pandas is used to organize saved result information into a tabular format. The result history can be retrieved from MySQL and displayed for inspection.

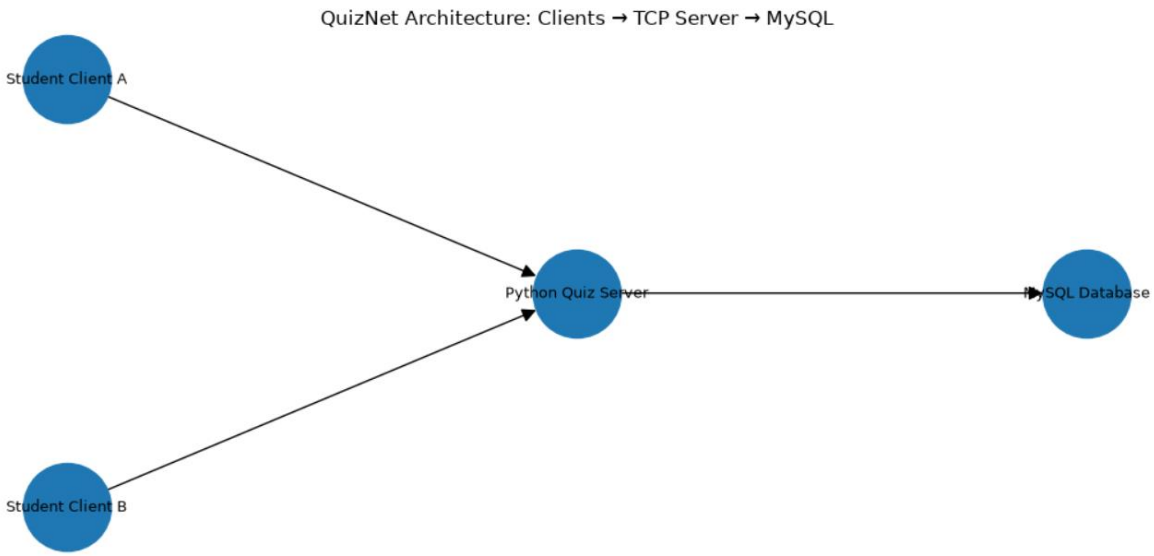
This makes it easier to review player names, scores, percentages, and completion times.

8.7 Network Visualization

NetworkX represents the main components of the application as nodes in a directed graph. Connections represent communication or data flow between clients, the server, and the database.

Matplotlib displays the graph. It can also be used to create student performance charts when result data is available.

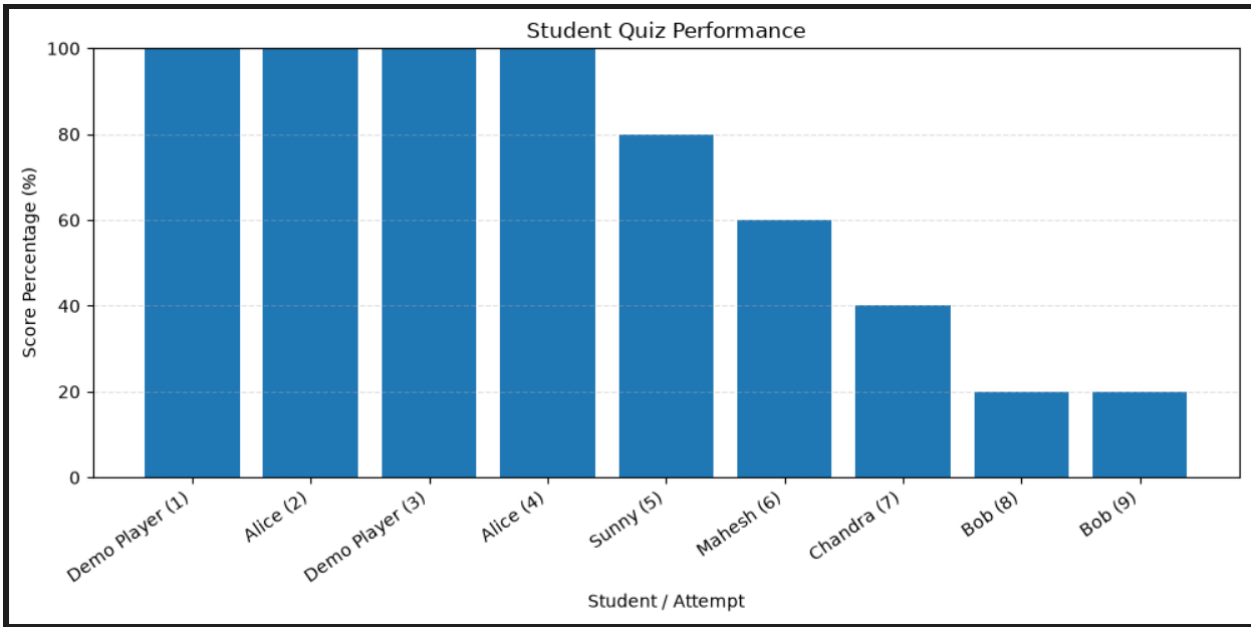
The architecture graph represents the logical design of the system; it is not a live network traffic monitor.



9. Testing and Results

Testing is required to verify that the system's components work as intended. The following table should be completed after executing the relevant notebook cells.

Test case	Expected outcome
Database setup	Database tables are available
Add question	New question is inserted
View questions	Stored questions are displayed
Single-client quiz	Quiz completes and a score appears
Multiple clients	Multiple clients can complete the
Leaderboard	Saved results are displayed
Architecture graph	NetworkX graph is displayed
Performance visualization	Student performance chart is displayed



	player_name	score	total_questions	percentage
0	Demo Player	5	5	100.0
1	Alice	5	5	100.0
2	Demo Player	5	5	100.0
3	Alice	5	5	100.0
4	Sunny	4	5	80.0
5	Mahesh	3	5	60.0
6	Chandra	2	5	40.0
7	Bob	1	5	20.0
8	Bob	1	5	20.0

9.1 Results Discussion

The project is designed to demonstrate communication between clients and a central server, concurrent client handling, database storage, and result visualization.

The final evaluation should be based on the actual notebook output. Screenshots of the executed cells should be included to demonstrate database initialization, quiz completion, leaderboard display, and visualizations where available.

Note: No numerical performance results or successful test outcomes are claimed here because those results must be confirmed from the actual execution of the notebook.

10. Advantages

The main advantages of QuizNet are:

- It supports multiple quiz participants through a centralized server.
- It demonstrates practical TCP socket programming.
- It uses threading to manage concurrent client connections.
- It stores questions and results persistently.
- It supports leaderboard and result-history functionality.
- It combines networking, database management, and visualization.
- Its notebook-based implementation makes individual steps easier to understand and demonstrate.

11. Limitations

The current implementation has several limitations:

- All players use the same set of questions.
- A login system and separate administrator account are not implemented.
- Network communication is not encrypted.
- The application is intended for local testing and demonstration.
- Client disconnections and additional validation could be handled more robustly.

12. Conclusion

QuizNet is a Python-based multiplayer quiz management system that combines networking, multithreading, database management, and visualization. TCP sockets enable communication between clients and the server, while threading helps the server handle multiple participants concurrently.

MySQL provides persistent storage for quiz questions and results. Pandas helps organize result data, while NetworkX and Matplotlib support system architecture and performance visualization.

The project provides practical experience in designing and implementing a client-server application. It also demonstrates how different programming and database technologies can be integrated into one system.

The project can be extended with authentication, additional quiz categories, improved security, and enhanced performance reporting.